

DD: FUL-02-FRAMEWORK — Camunda Primitives and Cross-Step Mechanisms

Developer implementation guide: DD_FUL-02-FRAMEWORK-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_FUL-02-FRAMEWORK-v1.0.md.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md The cross-cutting machinery used by every step satellite: how FUL-02 talks to Camunda (REST API + BPMN XML), how cancellation interrupts any step, how phase projection works, how operator BPMNs compose, the shared schemas every step touches, the v1.0 stubs every step depends on.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/06_fulfillment/DD_FUL-02-FRAMEWORK-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- GET /api/groups/{group}/members?operatorCode=<>
- GET /engine-rest/incident
- GET /engine-rest/process-instance/cam_pi_01HZX.../variables/expectedPaymentAmount
- GET /engine-rest/process-instance/{id}
- POST /api/notify/{notification-type}
- POST /api/orders
- POST /api/orders/{id}/cancel

- POST /api/orders/{orderId}/user-tasks/{taskId}/complete
- POST /engine-rest/deployment/create
- POST /engine-rest/external-task/ext_t_01HZX.../complete
- POST /engine-rest/external-task/ext_t_01HZX.../failure
- POST /engine-rest/external-task/fetchAndLock
- POST /engine-rest/message
- POST /engine-rest/process-definition/key/OrderCapture_WIK/start
- POST /engine-rest/task/{id}/complete

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- compute-expected-payment
- create-install-wo
- create-subscription
- kenya-kyc-leaders
- kenya-l1-kyc
- kyc-l1-supervisor-approval
- kyc-team-leader-approval
- main-start
- notification-type
- notify-success-end
- order-cascade-cancellation
- payment-timeout
- trigger-activation
- validate-order
- wait-install-finalize
- wait-payment

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- PaymentReceived

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md

The cross-cutting machinery used by every step satellite: how FUL-02 talks to Camunda (REST API + BPMN XML), how cancellation interrupts any step, how phase projection works, how operator BPMNs compose, the shared schemas every step touches, the v1.0 stubs every step depends on.

1. Camunda primitives

Every interaction between FUL-02 and Camunda is one of these primitives. The step satellites cite back into this section by anchor.

1.1 Start a process instance

When `POST /api/orders` succeeds envelope validation, FUL-02 starts a process instance:

```
POST /engine-rest/process-definition/key/OrderCapture_WIK/start
```

Authorization: Basic <module-svc-creds>
Content-Type: application/json

```
{
  "businessKey": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "variables": {
    "orderId": { "value": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1", "type": "String" },
    "customerId": { "value": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6", "type": "String" },
    "operatorCode": { "value": "WIK", "type": "String" },
    "packageRef": { "value": "pkg_INET_VOIP_RES", "type": "String" },
    "homepassId": { "value": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P", "type": "String" },
    "salesChannel": { "value": "FIELD_AGENT_PWA", "type": "String" },
    "salesAgent": { "value": "agent_01HZX9K8M7Q2N4P6R3T5W8Y0Z1", "type": "String" },
    "capturedAt": { "value": "2026-05-18T09:00:00Z", "type": "String" },
    "idempotencyKey": { "value": "fapwa-iphone15-20260518T090000-001", "type": "String" }
  }
}
```

Response 200:

```
{
  "id": "cam_pi_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "definitionId": "OrderCapture_WIK:5:abc-def-...",
  "businessKey": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "ended": false,
  "suspended": false
}
```

FUL-02 stores id onto customer_order.process_instance_id.

1.2 Worker long-poll (fetchAndLock)

Each worker bean runs a long-poll loop. All 16 v1.0 topics declared:

POST /engine-rest/external-task/fetchAndLock

```
{
  "workerId": "fulfillment-order-capture-worker-1",
  "asyncResponseTimeout": 30000,
  "maxTasks": 10,
  "topics": [
    { "topicName": "order-validate", "lockDuration": 5000 },
    { "topicName": "order-create-subscription", "lockDuration": 15000 },
    { "topicName": "order-compute-expected-payment", "lockDuration": 10000 },
    { "topicName": "order-notify-payment-instructions", "lockDuration": 5000 },
    { "topicName": "order-create-install-wo", "lockDuration": 30000 },
    { "topicName": "order-trigger-activation", "lockDuration": 95000 },
    { "topicName": "order-cascade-cancellation", "lockDuration": 30000 },
    { "topicName": "order-mark-rejected", "lockDuration": 5000 },
    { "topicName": "order-mark-cancelled", "lockDuration": 5000 },
    { "topicName": "order-mark-activation-failed", "lockDuration": 5000 },
    { "topicName": "order-mark-install-failed", "lockDuration": 5000 },
    { "topicName": "order-notify-success", "lockDuration": 5000 },
    { "topicName": "order-notify-failure", "lockDuration": 5000 }
  ]
}
```

Response 200 (when a task becomes available):

```
[
  {
    "id": "ext_t_01HZX...",
    "topicName": "order-validate",
    "processInstanceId": "cam_pi_01HZX...",
    "businessKey": "ord_01HZX...",
    "variables": {
      "orderId": { "value": "ord_01HZX...", "type": "String" },
      "customerId": { "value": "cust_01HX...", "type": "String" },
      "operatorCode": { "value": "WIK", "type": "String" },
      "packageRef": { "value": "pkg_INET_VOIP_RES", "type": "String" },
      "homepassId": { "value": "hp_01HX...", "type": "String" }
    },
    "lockExpirationTime": "2026-05-18T09:00:05Z"
  }
]
```

1.3 Worker complete + failure

Success:

POST /engine-rest/external-task/ext_t_01HZX.../complete

```
{
  "workerId": "fulfillment-order-capture-worker-1",
  "variables": {
    "validationOutcome": { "value": "PASS", "type": "String" },
    "validationDetails": { "value": "[{\"check\":\"\"HOMEPASS_SELLABLE\\\", \"result\":\"\"PASS\\\", ...}\",
                          "type": "Json" }
  }
}
```

Failure:

POST /engine-rest/external-task/ext_t_01HZX.../failure

```
{
  "workerId": "fulfillment-order-capture-worker-1",
  "errorMessage": "PACKAGE_NOT_AVAILABLE_IN_HOMEPASS_REGION",
  "errorDetails": "Package pkg_TV_HFC_BASIC requires tech regions [HFC]; HomePass hp_01HX... is in [GPON]",
  "retries": 0,
  "retryTimeout": 0
}
```

retries: 0 immediately raises an incident (visible via GET /engine-rest/incident). Non-zero leaves the task to be re-fetched after retryTimeout ms.

1.4 Message correlation

External signal → Camunda parked instance:

POST /engine-rest/message

```
{
  "messageName": "PaymentReceived",
  "businessKey": "ord_01HZX...",
  "processVariables": {
    "paymentId": { "value": "pmt_01HZ...", "type": "String" },
    "paidAmount": { "value": 15000.00, "type": "Double" },
    "paidCurrency": { "value": "KES", "type": "String" },
    "paymentMethod": { "value": "MPESA", "type": "String" }
  }
}
```

If no parked instance matches: 400 MismatchingMessageCorrelationException. Consumers DLQ original event to sophix.orders.message-correlation-dlq.

1.5 BPMN snippet — External Task

```
<bpmn:serviceTask id="validate-order" name="Validate Order"
  camunda:type="external"
  camunda:topic="order-validate">
  <bpmn:incoming>flow-start-to-validate</bpmn:incoming>
  <bpmn:outgoing>flow-validate-to-gateway</bpmn:outgoing>
</bpmn:serviceTask>
```

1.6 BPMN snippet — User Task with HTTP notification connector

The user task notifies the responsible team via Camunda's built-in HTTP connector, fired on task creation:

```
<bpmn:userTask id="kyc-l1-supervisor-approval" name="L1 KYC Approval"
  camunda:candidateGroups="kenya-l1-kyc">
  <bpmn:extensionElements>
    <camunda:taskListener event="create">
      <camunda:connector>
        <camunda:connectorId>http-connector</camunda:connectorId>
        <camunda:inputOutput>
          <camunda:inputParameter name="url">
            https://fulfillment-order-capture.sophix.internal/api/notify/kyc-approval-needed
          </camunda:inputParameter>
          <camunda:inputParameter name="method">POST</camunda:inputParameter>
          <camunda:inputParameter name="headers">
            <camunda:map>
              <camunda:entry key="Content-Type">application/json</camunda:entry>
            </camunda:map>
          </camunda:inputParameter>
          <camunda:inputParameter name="payload">
            {
              "taskId": "${task.id}",
              "taskName": "${task.name}",
              "businessKey": "${task.execution.businessKey}",
              "orderId": "${orderId}",
              "customerId": "${customerId}",
              "operatorCode": "${operatorCode}",
            }
          </camunda:inputParameter>
        </camunda:inputOutput>
      </camunda:connector>
    </camunda:taskListener>
  </bpmn:extensionElements>
</bpmn:userTask>
```

```

        "candidateGroup": "kenya-l1-kyc",
        "level": "L1",
        "channels": ["email", "sms"],
        "template": "kyc-approval-l1",
        "urgency": "medium"
    }
    </camunda:inputParameter>
</camunda:inputOutput>
</camunda:connector>
</camunda:taskListener>
</bpmn:extensionElements>
</bpmn:userTask>

```

The handler endpoint pattern (/api/notify/*) lives in FUL-02's NotificationHandlerResource — contract in §3 below.

1.7 BPMN snippet — Message catch event with boundary timer

Park-on-async-signal-with-timeout pattern, used by every message catch in v1.0:

```

<bpmn:intermediateCatchEvent id="wait-payment" name="Awaiting Payment">
  <bpmn:incoming>flow-notify-pay-instructions-to-wait</bpmn:incoming>
  <bpmn:outgoing>flow-payment-received-to-kyc-l1</bpmn:outgoing>
  <bpmn:messageEventDefinition messageRef="PaymentReceivedMessage" />
</bpmn:intermediateCatchEvent>

<bpmn:boundaryEvent id="payment-timeout" name="Payment Timeout"
  attachedToRef="wait-payment"
  cancelActivity="true">
  <bpmn:outgoing>flow-payment-timeout-to-cascade-cancel</bpmn:outgoing>
  <bpmn:timerEventDefinition>
    <bpmn:timeDuration xsi:type="bpmn:tFormalExpression">PT7D</bpmn:timeDuration>
  </bpmn:timerEventDefinition>
</bpmn:boundaryEvent>

<bpmn:message id="PaymentReceivedMessage" name="PaymentReceived" />

```

Operator BPMNs set their own durations. Step satellites document per-step timer durations.

1.8 BPMN snippet — Cancellation as interrupting boundary

Order cancellation can come at any phase. The BPMN models this with an interrupting boundary message event on the top-level subprocess:

```

<bpmn:subProcess id="order-main-flow">
  <bpmn:startEvent id="main-start" />
  <!-- all the order tasks and gateways -->
  <bpmn:endEvent id="main-end" />
</bpmn:subProcess>

<bpmn:boundaryEvent id="cancel-requested" name="Cancellation Requested"
  attachedToRef="order-main-flow"
  cancelActivity="true">
  <bpmn:outgoing>flow-cancel-to-cascade</bpmn:outgoing>
  <bpmn:messageEventDefinition messageRef="OrderCancellationRequestedMessage" />
</bpmn:boundaryEvent>

<bpmn:serviceTask id="cascade-cancellation"
  camunda:type="external"
  camunda:topic="order-cascade-cancellation" />

<bpmn:message id="OrderCancellationRequestedMessage" name="OrderCancellationRequested" />

```

POST /api/orders/{id}/cancel correlates OrderCancellationRequested; the boundary interrupts whatever subprocess was doing and routes to order-cascade-cancellation (owned by STEP-CANCELLATION). Works regardless of which step the order was at.

1.9 Variable read API

When a Kafka consumer needs to read a process variable (e.g., expectedPaymentAmount for the partial-payment threshold check in STEP-PAYMENT):

```
GET /engine-rest/process-instance/cam_pi_01HZX.../variables/expectedPaymentAmount
```

Response:

```
{ "value": 15000.00, "type": "Double" }
```

1.10 Deployment lifecycle

BPMN files live under `modules/fulfillment-order-capture/src/main/resources/workflows/`. On module startup, `BpmnDeploymentBootstrap` scans the folder and deploys each:

```
POST /engine-rest/deployment/create
Content-Type: multipart/form-data; boundary=---boundary

----boundary
Content-Disposition: form-data; name="deployment-name"
fulfillment-order-capture-startup-20260518
----boundary
Content-Disposition: form-data; name="deployment-source"
fulfillment-order-capture
----boundary
Content-Disposition: form-data; name="OrderCapture_WIK.bpmn"; filename="OrderCapture_WIK.bpmn"
Content-Type: application/octet-stream

<?xml version="1.0" encoding="UTF-8"?>
<bpmn:definitions ...>...</bpmn:definitions>
----boundary--
```

Camunda assigns each deployed BPMN a version. In-flight instances complete on the version they started on; new instances use the latest.

2. Per-operator BPMN composition

One BPMN per operator deployment. Files live in `modules/fulfillment-order-capture/src/main/resources/workflows/`. Naming convention: `OrderCapture_{OPERATOR_CODE}.bpmn`.

v1.0 ships: `OrderCapture_WIK.bpmn`, `OrderCapture_WUG.bpmn`, `OrderCapture_WTZ.bpmn`, `OrderCapture_YASSN.bpmn`.

Each operator's BPMN composes workers in any sequence, omits workers it doesn't need, defines its own user tasks and message catches. Workers are primitives owned by step satellites; the BPMN orchestrates. Adding a new operator: drop the BPMN file, add a `ful_02_config` row, configure Drools, configure candidate groups in `FOUNDATION_AUTH`. Zero new worker code in the common case.

Gateway decisions needing operator-specific logic are evaluated by Drools: a worker preceding the gateway calls Drools, sets a process variable (e.g., `payFirstRequired`), the BPMN gateway routes on the variable. Keeps BPMN free of operator-specific JUEL.

Camunda's deployment versioning: in-flight instances complete on the version they started on; new instances use the latest. BPMN bug fix never disrupts in-flight orders.

3. Notification handler endpoint contract

User tasks across all step satellites use the HTTP connector pattern from §1.6, POSTing to `FUL-02's NotificationHandlerResource`. One endpoint per task type, but a shared contract.

Endpoint pattern: `POST /api/notify/{notification-type}` where `notification-type` is set by each step satellite. v1.0 endpoints (referenced from step satellites): `/api/notify/kyc-approval-needed (STEP-KYC)`.

Common payload fields:

```
{
  "taskId":           "<Camunda user task ID>",
  "taskName":         "<BPMN userTask name attribute>",
  "businessKey":      "<orderId>",
  "orderId":          "<orderId>",
  "customerId":       "<customerId>",
  "operatorCode":     "WIK | WUG | WTZ | YASSN",
  "candidateGroup":   "<FOUNDATION_AUTH group code>",
  "channels":         ["email", "sms", "in_app_push"],
  "template":         "<staff-notification template name>",
  "urgency":          "low | medium | high",
  "<step-specific fields per satellite>"
}
```

Handler behavior (`NotificationHandlerResource`):

1. Verify the request is from Camunda (via shared secret header `X-Camunda-Connector-Secret`).

2. Resolve candidateGroup members via FOUNDATION_AUTH GET /api/groups/{group}/members?operatorCode=<>.
3. For each member: dispatch the notification via the staff-notification path.
4. v1.0 dispatch: write a row to staff_notification_stub with dispatch_method = INTERNAL_SMTP; send minimal email via internal SMTP wrapper.
5. Post-ICN-01: replace staff_notification_stub path with ICN-01 client (email + Slack + Teams + in-app push to BO UI).
6. Respond 200 to the Camunda connector immediately (best-effort dispatch — staff also see the task in their BO UI queue).

DD_NOT-01 is NEVER called for staff notifications. That module is strictly customer-scoped.

User task completion is the inverse path: the BO UI calls FUL-02's POST /api/orders/{orderId}/user-tasks/{taskId}/complete (step-specific endpoint contracts in each step satellite), FUL-02 verifies actor's candidate-group membership, forwards POST /engine-rest/task/{id}/complete with decision variables.

4. Phase projection

customer_order.current_phase is a coarse projection of Camunda's current_activity_id via ful_02_config.activity_phase_map (per-operator). The phase enum is the UNION across operators.

PhaseProjector (Camunda history listener) subscribes to Camunda's history stream, on each activity transition updates customer_order.current_phase + customer_order.current_activity_id, appends to customer_order_phase_history.

Unmapped activities → IN_PROGRESS_UNCATEGORIZED + emit warning event
sophix.orders.unmapped-activity-warning for ops.

Projection is for UI / analytics / customer-read. Camunda is the source of truth for runtime state — any consumer needing fine-grained activity-level state queries Camunda directly via GET /engine-rest/process-instance/{id}.

5. Shared schemas

Schemas touched by multiple step satellites. Per-step columns are documented in each step satellite's "Columns touched" section.

5.1 customer_order

The order aggregate. One row per order. Linked one-to-one with a Camunda process instance.

Column	Type	Notes
order_id	TEXT PK	ULID, server-generated. Example: ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
customer_id	TEXT NOT NULL	References ILM-CFG-01 (or customer_stub v1.0). Example: cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6
operator_code	TEXT NOT NULL	Operator tenant. Values: WIK / WUG / WTZ / YASSN
package_ref	TEXT NOT NULL	Package being ordered. Example: pkg_INET_V0IP_RES
homepass_id	TEXT NOT NULL	HomePass at install location. Example: hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P
sales_channel	TEXT NOT NULL	Enum. Values defined operator-wide: FIELD_AGENT_PWA / BACKOFFICE / CALL_CENTER / SELF_SERVICE. v1.0 KE deployment uses the first three; SELF_SERVICE reserved for future operator BPMNs.
sales_agent	TEXT NULL	Agent ULID for agent channels; NULL when self-service is used. Example: agent_01HZX9K8M7Q2N4P6R3T5W8Y0Z1

Column	Type	Notes
idempotency_key	TEXT NOT NULL	Channel-supplied dedup key. Example: fapwa-iphone15-20260518T090000-001
payment_reference	TEXT NULL	Reference passed to payment gateway; equals order_id by convention. Example: ord_01HZ...A
intended_activation_date	DATE NULL	Optional future date. Example: 2026-06-01
process_instance_id	TEXT NULL	Camunda process instance ID. NULL only if rejected at entry. Example: cam_pi_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
process_definition_key	TEXT NOT NULL	Camunda process key. Derived: OrderCapture_{operator_code}. Example: OrderCapture_WIK
current_phase	TEXT NOT NULL	Projection from Camunda activity. Values used by KE WIK BPMN v1.0: CAPTURED, IN_VALIDATION, AWAITING_PAYMENT, IN_KYC, READY_FOR_INSTALL, INSTALLING, ACTIVATING, ACTIVATED, ACTIVATION_FAILED, INSTALL_FAILED, CANCELLED, REJECTED. The column is TEXT (not enum) so future operator BPMNs can introduce new phase values without DDL changes.
current_activity_id	TEXT NULL	Raw Camunda activity ID. Examples: validate-order, wait-payment, kyc-l1-supervisor-approval, create-install-wo, wait-install-finalize, trigger-activation
subscription_id	TEXT NULL	FK to SUB-LM-01. Set by STEP-SUBSCRIPTION's worker. Example: sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4P
work_order_id	TEXT NULL	FK to WO module for install WO. Set by STEP-INSTALL's worker. Example: wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
validation_outcomes	JSONB NULL	Array of {check, outcome, detail} from STEP-VALIDATE. Example: [{"check": "HOMEPASS_SELLABLE", "outcome": "PASS"}, {"check": "PACKAGE_ACTIVE", "outcome": "PASS"}]
expected_payment_amount	NUMERIC(18,2) NULL	Snapshot at order time. Set by STEP-PAYMENT. Example: 15000.00
expected_payment_currency	TEXT NULL	ISO 4217. Example: KES
payment_received_amount	NUMERIC(18,2) NOT NULL DEFAULT 0	Cumulative paid. Set by STEP-PAYMENT. Example: 15000.00
equipment_bindings_captured	JSONB NULL	From WorkOrderFinalized message. Set by STEP-INSTALL. Example: [{"equipmentRequirement": "GPON_ONT", "equipmentRef": "ont_01HZ...", "serialNumber": "HW123456"}]
activation_correlation_id	TEXT NULL	Generated by STEP-ACTIVATION. Example: act-ord_01HZ...A-20260518T143000
activated_at	TIMESTAMP NULL	Set when phase reaches ACTIVATED. Example: 2026-05-18 14:35:12
failure_reason	TEXT NULL	Set on terminal failure by step satellite's mark-* worker. v1.0 KE WIK values: HOMEPASS_NOT_SELLABLE, PACKAGE_NOT_ACTIVE, PACKAGE_NOT_AVAILABLE_IN_HOMEPASS_REGION, DUPLICATE_SUBSCRIPTION, CUSTOMER_NOT_ELIGIBLE, KYC_REJECTED_L1, KYC_REJECTED_TEAM_LEADER, PAYMENT_TIMEOUT, INSTALL_TIMEOUT, WORK_ORDER_CANCELLED, ACTIVATION_FAILED_BILLING, ACTIVATION_FAILED_FULFILLMENT, FULFILLMENT_SAGA_TIMEOUT, CUSTOMER_REQUESTED, OPS_INITIATED, FRAUD_HOLD, OPERATOR_DECISION. Column is TEXT so future operator BPMNs can extend without DDL changes.

Column	Type	Notes
failure_detail	TEXT NULL	Free-text. Example: BIL-01 returned 400 INSUFFICIENT_BALANCE – credit balance 12000.00 KES, charge amount 15000.00 KES
created_at	TIMESTAMP NOT NULL DEFAULT NOW()	Example: 2026-05-18 09:00:00.123
created_by	TEXT NOT NULL	Agent ULID or system actor. Example: agent_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
last_updated_at	TIMESTAMP NOT NULL DEFAULT NOW()	Updated on every phase change
metadata	JSONB NULL	Channel payload. Example: {"capturedLocation":{"lat":-1.286,"lng":36.817},"deviceId":"iphone15-FA001","notes":"Saturday install"}

Indexes: PK; UNIQUE (operator_code, sales_channel, idempotency_key); (customer_id, created_at DESC); (operator_code, current_phase, created_at DESC); partial (process_instance_id) WHERE process_instance_id IS NOT NULL; partial (subscription_id) WHERE subscription_id IS NOT NULL; partial (work_order_id) WHERE work_order_id IS NOT NULL; (payment_reference). P10Y retention.

Sample data:

order_id	operator	package	channel	proc_def_key	current_phase	sub_id	wo_id
ord_01HZ...A	WIK	pkg_INET_VOIP_RES	FIELD_AGENT_PWA	OrderCapture_WIK	ACTIVATED	sub_01HX...A	wo_01HZ...A
ord_01HZ...B	WIK	pkg_INET_100M_ONLY	SELF_SERVICE	OrderCapture_WIK	AWAITING_PAYMENT	sub_01HX...B	NULL
ord_01HZ...C	WIK	pkg_INET_VOIP_RES	CALL_CENTER	OrderCapture_WIK	INSTALLING	sub_01HX...C	wo_01HZ...C
ord_01HZ...D	WIK	pkg_TV_HFC_BASIS	BACKOFFICE	OrderCapture_WIK	REJECTED	NULL	NULL
ord_01HZ...E	YASSN	pkg_INET_50MSN	FIELD_AGENT_PWA	OrderCapture_YASSN	READY_FOR_INSTALL	sub_01HX...E	NULL
ord_01HZ...F	WIK	pkg_INET_VOIP_RES	BACKOFFICE	OrderCapture_WIK	IN_KYC	sub_01HX...F	NULL

5.2 customer_order_phase_history

Append-only history of phase transitions. Monthly partitioned. Written by PhaseProjector (\$4 mechanism).

Column	Type	Notes
id	TEXT PK	ULID. Example: ordh_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
order_id	TEXT NOT NULL	FK to customer_order. Example: ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
operator_code	TEXT NOT NULL	Denormalized for partition pruning. Values: WIK / WUG / WTZ / YASSN
previous_phase	TEXT NULL	NULL on initial CAPTURED entry. Example: AWAITING_PAYMENT
new_phase	TEXT NOT NULL	Same enum as customer_order.current_phase. Example: IN_KYC
previous_activity_id	TEXT NULL	Raw Camunda activity. Example: wait-payment
new_activity_id	TEXT NOT NULL	Raw Camunda activity. Example: kyc-l1-supervisor-approval
transition_at	TIMESTAMP NOT NULL DEFAULT NOW()	Example: 2026-05-18 11:23:00.512
triggered_by	TEXT NOT NULL	Values: worker_completion, message_correlation, timer_fired, user_task_completed, cancellation_requested, system_startup
triggered_by_detail	TEXT NULL	Detail. Examples: worker class (ValidateOrderWorker), message name (PaymentReceived), timer activity id (payment-timeout), user task id (kyc-l1-supervisor-approval), actor user ID

Column	Type	Notes
metadata	JSONB NULL	Transition-specific context. Example: {"paymentId": "pmt_01HZ...", "amount": 15000.00, "cumulativePaid": 15000.00}

Indexes: PK; (order_id, transition_at DESC); (operator_code, new_phase, transition_at DESC).
Monthly partitioned. P10Y retention.

Sample data (WIK happy-path order from parent §2.1):

id	previous_phase	new_phase	new_activity_id	triggered_by	at
ordh_01HZ...1	NULL	CAPTURED	main-start	system_startup	2026-05-18 09:00:00
ordh_01HZ...2	CAPTURED	IN_VALIDATION	validate-order	worker_completion	2026-05-18 09:00:01
ordh_01HZ...3	IN_VALIDATION	IN_VALIDATION	create-subscription	worker_completion	2026-05-18 09:00:02
ordh_01HZ...4	IN_VALIDATION	IN_VALIDATION	compute-expected-payment	worker_completion	2026-05-18 09:00:03
ordh_01HZ...5	IN_VALIDATION	AWAITING_PAYMENT	wait-payment	worker_completion	2026-05-18 09:00:03
ordh_01HZ...6	AWAITING_PAYMENT	IN_KYC	kyc-l1-supervisor-approval	message_correlation	2026-05-18 11:23:00
ordh_01HZ...7	IN_KYC	IN_KYC	kyc-team-leader-approval	user_task_completed	2026-05-18 11:35:00
ordh_01HZ...8	IN_KYC	READY_FOR_INSTALL	create-install-work	user_task_completed	2026-05-18 12:00:00
ordh_01HZ...9	READY_FOR_INSTALL	INSTALLING	wait-install-finalize	worker_completion	2026-05-18 12:00:05
ordh_01HZ...10	INSTALLING	ACTIVATING	trigger-activation	message_correlation	2026-05-18 14:30:00
ordh_01HZ...11	ACTIVATING	ACTIVATED	notify-success-end	worker_completion	2026-05-18 14:35:12

5.3 ful_02_config

Per-operator config. One row per operator.

Column	Type	Notes
operator_code	TEXT PK	Values: WIK / WUG / WTZ / YASSN
process_definition_key	TEXT NOT NULL	Camunda process key. Derived: OrderCapture_{operator_code}. Example: OrderCapture_WIK
bpmn_version_tag	TEXT NULL	Human-readable tag, set by BpmnDeploymentBootstrap. Example: 2026-05-15-v1
worker_lock_durations	JSONB NOT NULL	Map of topic → lock_ms. Example: {"order-validate": 5000, "order-create-subscription": 15000, "order-trigger-activation": 95000}
worker_retry_policy	JSONB NOT NULL	Map of topic → {retries, retryTimeoutMs}. Example: {"order-validate": {"retries": 0, "retryTimeoutMs": 0}, "order-create-install-work": {"retries": 3, "retryTimeoutMs": 300000}}
activity_phase_map	JSONB NOT NULL	Map of camunda_activity_id → order_phase. Example for WIK: {"validate-order": "IN_VALIDATION", "wait-payment": "AWAITING_PAYMENT", "kyc-l1-supervisor-approval": "IN_KYC", "wait-install-finalize": "INSTALLING", "trigger-activation": "ACTIVATING", "notify-success-end": "ACTIVATED"}
future_activation_lead_days	INTEGER NOT NULL DEFAULT 5	Lead time. Example: 5
default_billing_mode	TEXT NOT NULL	Passed to SUB-LM-01. Values: POSTPAID / PREPAID. Example: POSTPAID
default_cycle_model	TEXT NOT NULL	Values: CALENDAR_ANCHOR / ROLLING_DAYS. Example: CALENDAR_ANCHOR

Column	Type	Notes
updated_at	TIMESTAMP NOT NULL	Example: 2026-05-18 09:00:00
updated_by	TEXT NOT NULL	Example: admin_user_42

Sample data:

operator	process_key	bpmn_version	lead_days	billing_mode	cycle_model
WIK	OrderCapture_WIK	2026-05-15-v1	5	POSTPAID	CALENDAR_ANCHOR
WUG	OrderCapture_WUG	2026-05-15-v1	5	POSTPAID	CALENDAR_ANCHOR
WTZ	OrderCapture_WTZ	2026-05-15-v1	5	POSTPAID	CALENDAR_ANCHOR
YASSN	OrderCapture_YASSN	2026-05-15-v1	3	PREPAID	ROLLING_DAYS

6. v1.0 stubs

Three small stub tables that exist while their owning DDs are pending. Scheduled for removal as their owners land.

6.1 customer_stub

Minimal customer record until ILM-CFG-01 lands.

Column	Type	Notes
customer_id	TEXT PK	ULID. Example: cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6
name	TEXT NOT NULL	Example: John Mwangi
msisdn	TEXT NOT NULL	Example: +254712345678
email	TEXT NULL	Example: john.mwangi@example.com
id_document_ref	TEXT NULL	Reference to scanned ID. Example: idoc_01HX...
kyc_status	TEXT NOT NULL	Values: PENDING / L1_APPROVED / APPROVED / REJECTED. Example: APPROVED
operator_code	TEXT NOT NULL	Tenant. Values: WIK / WUG / WTZ / YASSN
created_at	TIMESTAMP NOT NULL	Example: 2026-05-18 09:00:00

6.2 install_wo_stub

Minimal install WO record + admin finalize endpoint until WO module DD lands. v1.0 supports kind=install only. Future WO module DD may add other kinds; the column is kept as TEXT (not enum) so future kinds can be added without DDL changes.

Column	Type	Notes
work_order_id	TEXT PK	ULID. Example: wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
order_id	TEXT NOT NULL	FK to customer_order. Example: ord_01HZX...
kind	TEXT NOT NULL	v1.0 value: install. The column accepts other values reserved for the future WO module DD; v1.0 workers only write install.
status	TEXT NOT NULL	Values: CREATED / FINALIZED / CANCELLED. Example: FINALIZED
equipment_bindings_jsonb	JSONB NULL	Set on FINALIZE. Example: [{"equipmentRequirement": "GPON_ONT", "equipmentRef": "ont_01HZ...", "serialNumber": "HW123456"}]
created_at	TIMESTAMP NOT NULL	Example: 2026-05-18 12:00:00
finalized_at	TIMESTAMP NULL	Example: 2026-05-18 14:30:00

6.3 staff_notification_stub

Minimal staff notification audit + internal SMTP wrapper until ICN-01 lands.

Column	Type	Notes
notification_id	TEXT PK	ULID. Example: notif_01HZX...
order_id	TEXT NOT NULL	FK to customer_order. Example: ord_01HZX...
task_id	TEXT NOT NULL	Camunda user task ID. Example: cam_task_l1_01HZ...
candidate_group	TEXT NOT NULL	v1.0 KE values: kenya-l1-kyc, kenya-kyc-leaders. Future operators add their own groups at deployment time.
payload_jsonb	JSONB NOT NULL	Full HTTP connector payload from §1.6
dispatch_method	TEXT NOT NULL	Values: INTERNAL_SMTP. Example: INTERNAL_SMTP
dispatch_status	TEXT NOT NULL	Values: DISPATCHED / FAILED. Example: DISPATCHED
created_at	TIMESTAMP NOT NULL	Example: 2026-05-18 11:23:01

7. Cross-step rules

Rules that apply across all step satellites. Per-step rules live in each step satellite.

ID	Rule	Triggered on
R-FUL-02-FW-1	Single primary entry: POST /api/orders (STEP-CAPTURE). Auth: channel ↔ role pairing — ORDER_FIELD_AGENT, ORDER_BACKOFFICE, ORDER_CALL_CENTER, ORDER_CUSTOMER_SELF. Mismatch → 403 CHANNEL_ROLE_MISMATCH. ORDER_BACKOFFICE_ADMIN gates admin endpoints.	Auth
R-FUL-02-FW-2	All workers use the External Task pattern (§1.2-§1.3).	All workers
R-FUL-02-FW-3	Workers are operator-agnostic. operatorCode is a process variable; behaviour varies via ful_02_config or Drools. No hardcoded operator branches.	Operator isolation
R-FUL-02-FW-4	Per-topic lock durations from ful_02_config.worker_lock_durations. Defaults documented in each step satellite.	Timeouts
R-FUL-02-FW-5	Worker idempotency: workers check customer_order columns to detect prior partial completion before redoing work that has side effects.	Idempotency
R-FUL-02-FW-6	Worker failure → §1.3 /failure with retries from ful_02_config.worker_retry_policy. Per-topic policies in each step satellite.	Failure
R-FUL-02-FW-7	Message correlation via §1.4. MismatchingMessageCorrelationException → DLQ to sophix.orders.message-correlation-dlq for finance/ops review. Consumers are idempotent (per-order processed-set keyed on event ID).	Async correlation
R-FUL-02-FW-8	Cancellation can come at any phase. BPMNs use the interrupting boundary message event on top-level subprocess (§1.8). Cancel API in STEP-CANCELLATION.	Cancellation
R-FUL-02-FW-9	User tasks notify via HTTP connector (§1.6) → NotificationHandlerResource (§3). v1.0 dispatch via staff_notification_stub. DD_NOT-01 NOT called for staff.	User task notification
R-FUL-02-FW-10	User task completion verifies actor's candidate-group membership in FOUNDATION_AUTH, then forwards POST /engine-rest/task/{id}/complete.	User task completion

ID	Rule	Triggered on
R-FUL-02-FW-11	current_phase is a coarse projection from Camunda activity via ful_02_config.activity_phase_map. PhaseProjector (Camunda history listener) writes customer_order.current_phase + customer_order.current_activity_id + appends customer_order_phase_history. Camunda is source of truth for runtime state.	Phase projection
R-FUL-02-FW-12	customer_order + customer_order_phase_history 10-year retention. Camunda history 90d hot. Phase history monthly partitioned (high write rate).	Audit / retention
R-FUL-02-FW-13	One BPMN per operator. Files in modules/fulfillment-order-capture/src/main/resources/workflows/. Naming OrderCapture_{OPERATOR_CODE}.bpmn. Deployed via \$1.10 on startup. Camunda's deployment versioning preserves in-flight instances.	BPMN deployment
R-FUL-02-FW-14	Gateway decisions needing operator-specific logic evaluated by Drools KJAR: preceding worker calls Drools, sets a process variable, BPMN gateway routes on the variable. Keeps BPMN free of operator-specific JUEL.	Operator decisions
R-FUL-02-FW-15	Each event emitted by step satellites carries full order context (subscriptionId if set, workOrderId if set, customerId, packageRef, homepassId, operatorCode, current phase, salesChannel, salesAgent, orderId).	Event consistency